WEEK 1 · DAY 4 · OVERVIEW

Autocorrelation

ACF and PACF



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Course 4's Day 11 drew the correlogram of a daily yield change and found it near zero at every lag.

Day 5 fits AR and ARIMA models to the five series' daily changes, and it needs two answers first.

Is there any memory in the changes worth a model, and if there is, how many lags back does it reach?

Today reads that memory on the development years, sets how large an autocorrelation must be to count, and draws the pictures that name a model's order.

A test fits nothing to score, so we read all of the development years; the held-out years stay closed.

OBJECTIVES

Compute an autocorrelation by hand and with statsmodels, draw an ACF with its confidence band, and measure what noise alone puts outside the band.

Read the ACF of a series' changes and of their squares, compute a partial autocorrelation as a regression coefficient, and read the identification grid to name an AR or MA order.

Run the Ljung-Box test by hand and with statsmodels, and check whether an autocorrelation holds across windows of the development years.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 4

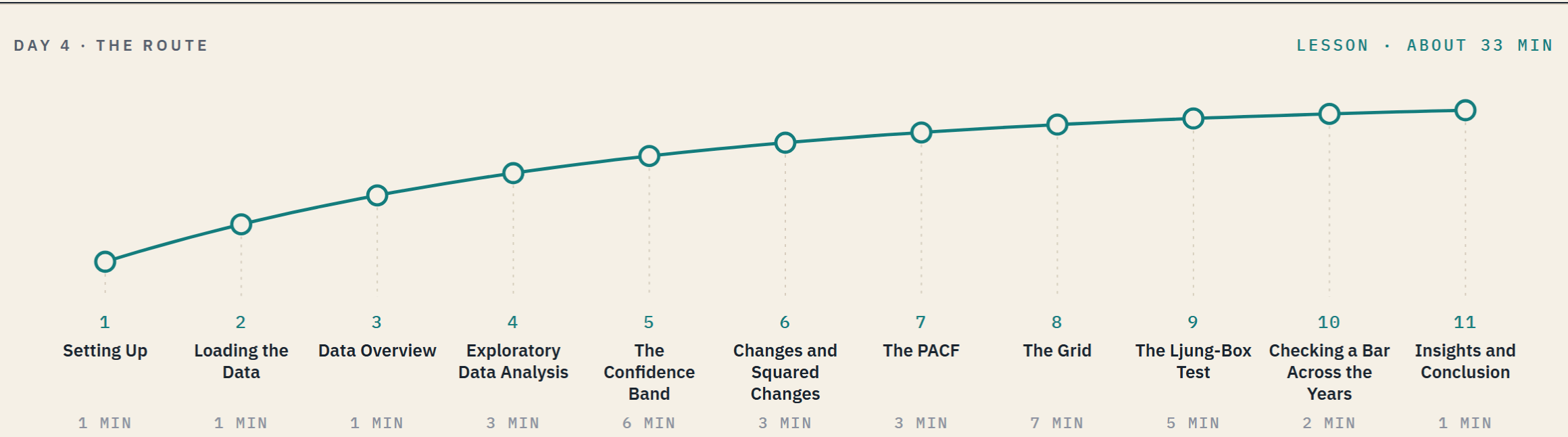


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 33 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy import stats
```

Read both files and build Day 2's series.

Loading the Data

BOTH FILES

```
yields = pd.read_csv(DATA + "treasury_par_yields.csv", parse_dates=["date"], index_col="date")
spreads = pd.read_csv(DATA + "course3_synthetic_spreads.csv", parse_dates=["date"], index_col="date")
CUT = "2022-12-30"
development = yields.loc[:CUT]
```

DAY 2'S FIVE SERIES

```
levels = pd.DataFrame({"2 Yr": (development["2 Yr"] * 100).round(2),
                       "10 Yr": (development["10 Yr"] * 100).round(2),
                       "2s10s": ((development["10 Yr"] - development["2 Yr"]) * 100).round(2),
                       "IG": spreads.loc[:CUT, "ig_spread_bp"],
                       "HY": spreads.loc[:CUT, "hy_spread_bp"]})
changes = levels.diff().dropna()
print(len(levels), "levels and", len(changes), "changes, from", changes.index[0].date(), "to", changes.index[-1].date())
```

```
2001 levels and 2000 changes, from 2015-01-05 to 2022-12-30
```

Check the shape and recall Course 4's lag.

Data Overview

THE SHAPE

```
print(changes.shape, "| blank cells:", changes.isna().sum().sum())
```

```
(2000, 5) | blank cells: 0
```

RECALLING COURSE 4'S LAG

```
print("Level:", round(levels["10 Yr"].autocorr(1), 4), "| Change:", round(changes["10 Yr"].autocorr(1), 4))
```

```
Level: 0.9977 | Change: -0.0075
```

The level copies the day before and the change sits near zero. Today finds out how near zero a number must be to count as zero.

Exploratory Data Analysis

the 10 Yr's first six daily changes, each less the mean of all 2,000 (0.09 bp)

date	change less mean	one row before	product
2015-01-05	-8.09	none	none
2015-01-06	-7.09	-8.09	**57.33**
2015-01-07	-1.09	-7.09	**7.71**
2015-01-08	6.91	-1.09	**-7.52**
2015-01-09	-5.09	6.91	**-35.17**
2015-01-12	-6.09	-5.09	**30.98**

… and so on down all 2,000 rows

lag 1 autocorrelation

$$\frac{\text{sum of the products}}{\text{sum of the squares of the first column}}$$

lag k: move the column down k rows

FIGURE 2 The 10 Yr's first six changes less their 0.09 bp mean, the column one row down, and each pair's product.

THREE WAYS, LAGS 1 TO 5

	BY HAND	ACF	AUTOCORR
1	-0.0075	-0.0075	-0.0075
2	-0.0372	-0.0372	-0.0373
3	-0.0172	-0.0172	-0.0172
4	-0.0008	-0.0008	-0.0008
5	0.0316	0.0316	0.0318

BY HAND

```
by_hand = [np.sum(centred[k:] * centred[:-k]) / np.sum(centred ** 2) for k in range(1, 6)]
```

Five numbers at a time is slow. The ACF draws every lag at once, and it needs a rule for which bars matter.

The Confidence Band

THE BAND

```
band = 1.96 / np.sqrt(len(values))
print("Band: plus or minus", round(band, 4))
```

```
Band: plus or minus 0.0438
```

PREDICT 1

Q. Of the 10 Yr change's 40 bars, lags 1 to 40, how many lie outside the band? Write down a number.

THE ACF WITH THE BAND

```
fig, ax = plt.subplots(figsize=(12, 4.5))
plot_acf(values, lags=40, zero=False, ax=ax)
ax.axhline(band, color="C1", linestyle="--")
```

ONE ACF AS BARS

```
def draw_acf(ax, bars, band):
    colours = ["C1" if abs(b) > band else "C0" for b in bars]
    ax.bar(range(1, len(bars) + 1), bars, color=colours, width=0.7)
    ax.axhspan(-band, band, color="C0", alpha=0.15)
    ax.axhline(0, color="black", linewidth=0.8)
```

That range is the confidence band: a bar outside it is larger than noise usually makes.

Compare each change's ACF with its square's.

Changes and Squared Changes

THE SQUARED CHANGE

```
squares = values ** 2
square_bars = acf(squares, nlags=40)[1:]
print("Lags 1, 5, 20, 40:", square_bars[[0, 4, 19, 39]].round(4))
print("Outside the band:", (np.abs(square_bars) > band).sum(), "of 40 | lowest bar:", round(square_bars.min(), 4))
print("Absolute change, outside the band:", (np.abs(acf(np.abs(values), nlags=40)[1:]) > band).sum(), "of 40")
```

EVERY SERIES

```
rows = []
for name in changes.columns:
    change_bars = acf(changes[name].to_numpy(), nlags=40)[1:]
    size_bars = acf(changes[name].to_numpy() ** 2, nlags=40)[1:]
```

PREDICT 2

Q. Sketch the ACF of the squared 10 Yr change to lag 40, and write down how many of its 40 bars lie outside the band.

A squared change keeps the size of a day's move and throws its sign away.

The Partial Autocorrelation

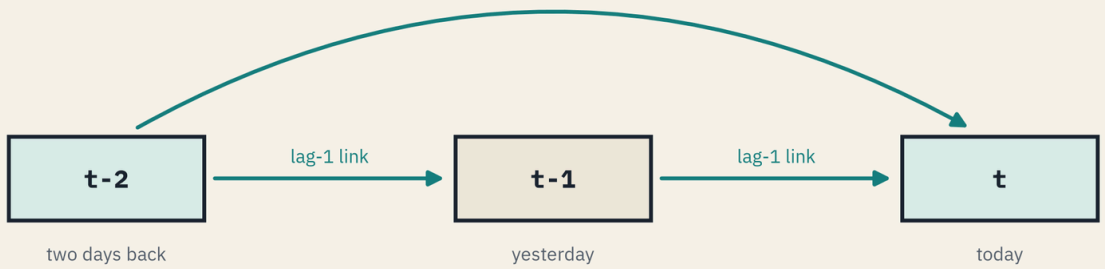


FIGURE 3 Three days in a row: the ACF at lag 2 counts both routes from t-2 to t, the PACF only the direct one.

TWO ROUTES TO LAG 2

A value two days back can reach today two ways: straight, or through yesterday.

ONLY THE DIRECT ONE

The **partial autocorrelation** at lag k keeps only the direct one: the link between today and k days back once the days between are known.

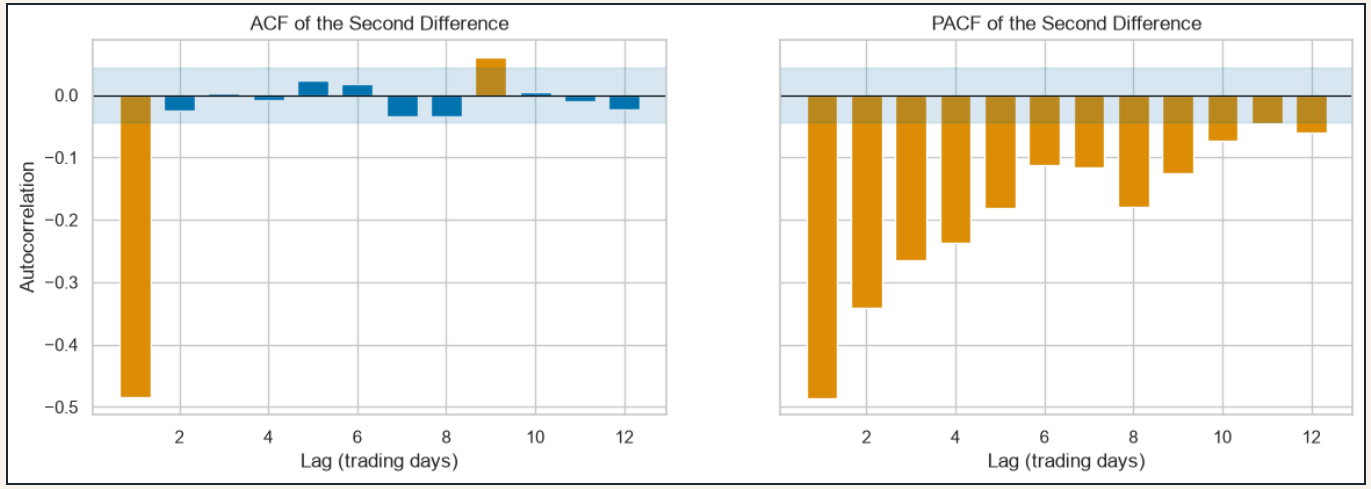
THE PACF AS A REGRESSION

```
rows_2 = pd.DataFrame({"change": ten, "lag 1": ten.shift(1), "lag 2": ten.shift(2)}).dropna()
ols = sm.OLS(rows_2["change"], sm.add_constant(rows_2[["lag 1", "lag 2"]])).fit()
```

```
Lag-2 coefficient of the regression: -0.03734831394052713
pacf at lag 2, OLS method:           -0.037348313940527136
```

The ACF shows all the memory; the PACF shows how many lags carry it directly. Together they name a model.

The Identification Grid



THE SECOND DIFFERENCE

```
ACF, lags 1 to 5:  [-0.485 -0.025  0.002 -0.009  0.024]
PACF, lags 1 to 5: [-0.485 -0.34  -0.265 -0.236 -0.181]
```

PREDICT 3

Q. An AR(1) with phi 0.7 has an autocorrelation of 0.7 at lag 1. Write down its autocorrelation at lag 3.

AN AR(1) WITH PHI 0.7

```
ar1 = ArmaProcess(ar=[1, -0.7], ma=[1])
ar1_values = ar1.generate_sample(nsample=2000, distrvs=rng.standard_normal, burnin=100)
```

Differencing once too often makes an MA(1) out of a series that had none.

The Ljung-Box Test

Q BY HAND

```
n = len(values)
lags_10 = np.arange(1, 11)
q_by_hand = n * (n + 2) * np.sum(bars[:10] ** 2 / (n - lags_10))
```

FROM STATSMODELS

```
acorr_ljungbox(values, lags=[10])
```

PREDICT 4

Q. At 10 lags, for how many of the five series' changes does Ljung-Box reject at 5 percent? Write down a number.

THE Q STATISTIC

Its **Q statistic** is n (n + 2) times the sum, over lags k from 1 to m, of the squared bar divided by n - k.

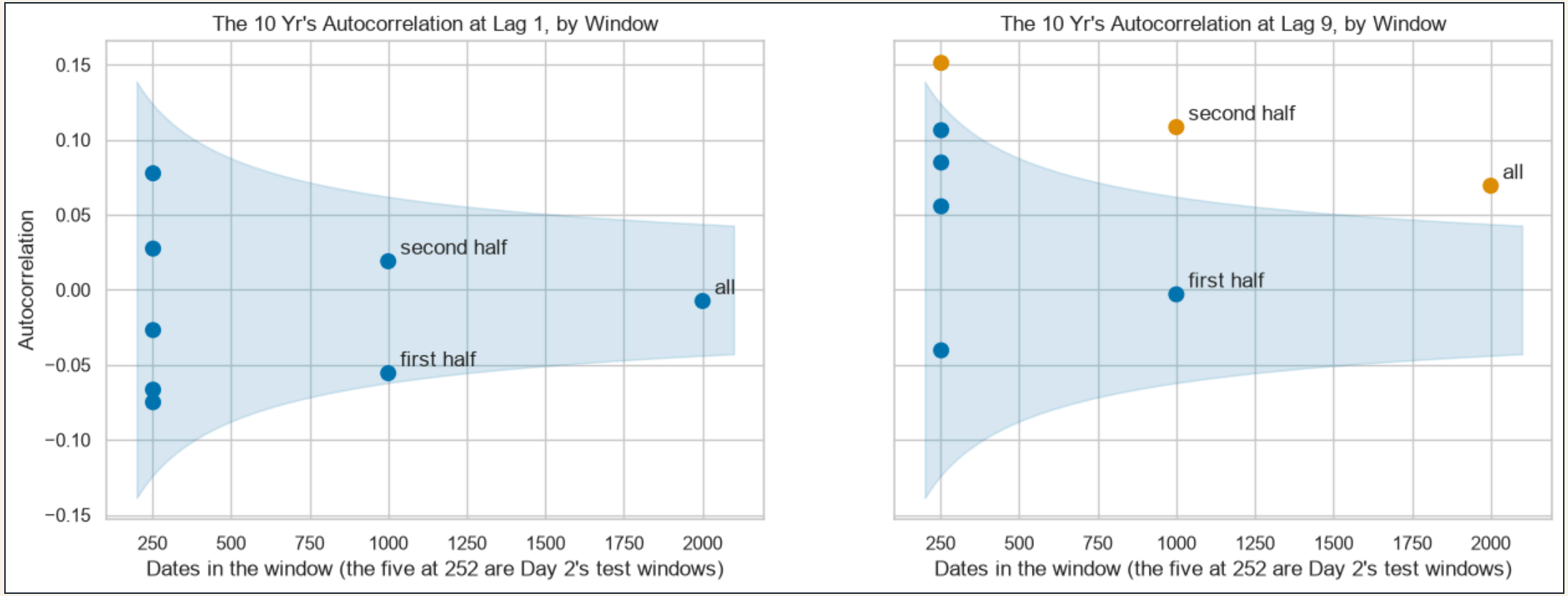
FOR FIVE SERIES

```
for name in changes.columns:
    for kind, series in [("change", changes[name]), ("square", changes[name] ** 2)]:
        test = acorr_ljungbox(series.to_numpy(), lags=[10, 20])
        rows.append({"series": name, "kind": kind, "p, 10 lags": test["lb_pvalue"].iloc[0],
```

Its null hypothesis: no autocorrelation at any lag up to m.

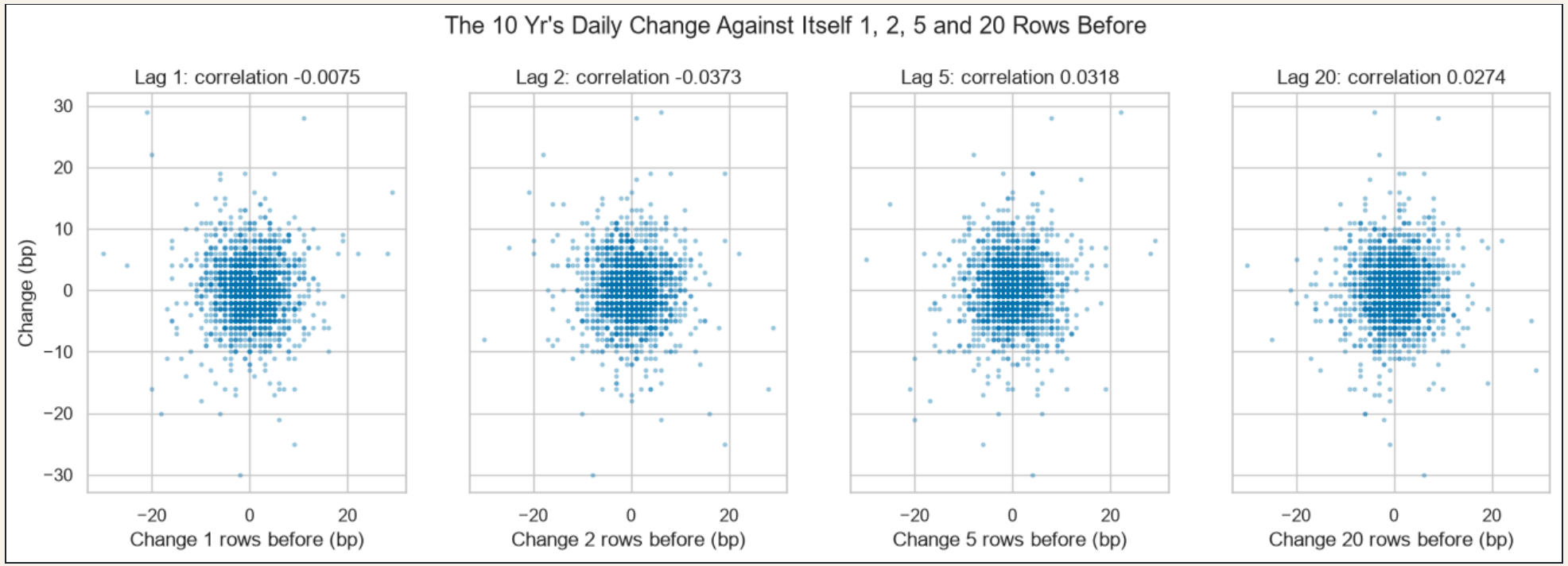
Read lags 1 and 9 window by window.

Checking a Bar Across the Years



The band is 1.96 / sqrt(n), so a shorter window has a wider band.

Insights and Conclusion



THE DAY IN TWO LINES

`acf` is the textbook autocorrelation; **`.autocorr`** is a close cousin with two means.

Day 5 fits AR and ARIMA models to these changes and scores them walk-forward against the last value carried forward.

You Can Now

- Compute an autocorrelation by hand and with **`acf`**, and say how **`.autocorr`** differs.
- Draw an ACF and a PACF with **`plot_acf`** and **`plot_pacf`**, and say which bars lie outside the band.
- Measure what white noise alone puts outside the band.
- Compute a partial autocorrelation as the last coefficient of a regression on lags.
- Simulate AR, MA and ARMA processes with **`ArmaProcess`** and read an order from the identification grid.
- Run the Ljung-Box test by hand and with **`acorr_ljungbox`**.
- Check whether an autocorrelation holds across windows of the development years.

NEXT
WEEK 1 · DAY 5

Case Study: AR and ARIMA on the Curve

Day 5 fits AR and ARIMA models and scores them against the last value.